

W05-asm1— Performance Report

Parallel Monte Carlo Evaluation of the Gamma Function (Leapfrog Method)

Test environment: CTCP cluster (it096843, Linux), 32 logical cores available (`nproc` = 32), so all tested configurations (`numproc` = 1, 2, 4, 8, 16) run **without core oversubscription**.

1. Accuracy

The program was run with **x = 5.0** and **N = 100,000,000** samples. Since $\Gamma(5) = 4! = 24$ exactly, this provides a direct check of estimator accuracy.

Metric	Value
True value $\Gamma(5)$	24.0000
Monte Carlo estimate	24.0026
Absolute error	0.0026
Relative error	$\approx 0.0108\%$

The estimate is identical across every `numproc` configuration tested (1, 2, 4, 8, 16), confirming that the **leapfrog method correctly preserves the statistical properties of the underlying LCG sequence** regardless of how many processes share it — all processes sample from the same pool of random numbers rather than from independent, uncorrelated sequences. A relative error of $\sim 0.01\%$ at $N = 10^8$ is consistent with the expected Monte Carlo convergence rate of $O(1/\sqrt{N})$.

2. Performance Results and Amdahl's Law

2.1 Raw timing data (average of 3 runs per configuration, seconds)

Note on methodology: f is *not* fitted using all five data points — it is back-solved algebraically from only the two endpoint measurements ($p=1$ and $p=16$), using $f = (p-S)/(S(p-1))$ derived from Amdahl's formula. The resulting Amdahl curve is then evaluated at every p for comparison, but this only tests whether the two endpoints are self-consistent with a single fixed f — it does not confirm that a fixed- f model actually fits the intermediate points ($p=2, 4, 8$) well.

numproc	avg total_time (s)	measured S(p)	Amdahl S(p) (curve through p=1 & p=16 only, $f \approx 0.0688$)	Efficiency
1	1.488487	1.000	1.000	1.000
2	0.757245	1.966	1.871	0.983
4	0.394361	3.774	3.316	0.944
8	0.228770	6.506	5.400	0.813
16	0.189020	7.875	7.875	0.492

$f \approx 0.0688$ is derived solely from $T(p=1)=1.488487\text{s}$ and $T(p=16)=0.189020\text{s}$ (measured $S(16) = 1.488487/0.189020 \approx 7.875$, solved for f). It is **not** an independent measurement of "serial code fraction", given that most of the $p=1 \rightarrow p=16$ gap is actually caused by `MPI_Reduce` overhead growing with p , not fixed serial computation.

Theoretical maximum speedup implied by this f : $S_\infty = 1/f \approx \mathbf{14.5}$ (again, this figure inherits the same two-point-fit caveat above).

2.2 Discussion — are the speed-ups what we would expect?

Overall, yes — this is much closer to textbook Amdahl's Law behaviour than the earlier (10-core, oversubscribed) test run. With 32 physical/ logical cores available and a maximum of 16 processes tested, there is no oversubscription anywhere in this dataset, so the results reflect genuine parallel scaling rather than OS scheduling artifacts.

Up to $p = 4$, efficiency stays high (94–98%). This is exactly what we'd expect from an **embarrassingly parallel** workload — each Monte Carlo sample is computed completely independently, and the only communication in the leapfrog design is a handful of `MPI_Send` calls at start-up plus a single `MPI_Reduce` at the end.

At $p = 8$, efficiency has already declined moderately to 81.3%, and at $p = 16$ it drops sharply to 49.2% — even though there is no core oversubscription at either point (32 cores were available). Looking at the `reduce_time` column explains why:

numproc	reduce_time (avg, s)	reduce_time as % of total_time
1	0.0000330	0.002%
2	0.0014	0.18%
4	0.00186	0.47%
8	0.0069	3.0%
16	0.0402	21.3%

The cost of `MPI_Reduce` grows roughly two orders of magnitude from $p=1$ to $p=16$, and by $p=16$ it consumes over a fifth of the total run time. This is a genuine, measurable overhead that **grows with p** — unlike Amdahl's Law's assumption of a fixed serial fraction f independent of p . The fitted $f \approx 0.0688$ — again, back-solved only from the $p=1$ and $p=16$ endpoints, not fitted to all five points — is therefore best understood as capturing this **growing reduction/synchronisation overhead between those two specific measurements**, rather than an inherent serial portion of the Monte Carlo algorithm itself (the underlying integration is close to 0% serial by construction). The fact that the "Amdahl $S(p)$ " column consistently *underestimates* the actual measured speedup at the intermediate points (e.g. predicted 5.400 vs. measured 6.506 at $p=8$) is itself evidence that a single fixed f does not describe this system well across the full range — the true overhead is p -dependent, not constant as Amdahl's simple model assumes.

Conclusion on speed-up: the results are close to ideal for $p \leq 4$ (94%+ efficiency), which is what should be expected for an embarrassingly parallel Monte Carlo computation. Efficiency then declines progressively at $p=8$ (81.3%) and more sharply at $p=16$ (49.2%). This drop-off is attributable to `MPI_Reduce` overhead scaling with process count rather than any algorithmic serial bottleneck or hardware oversubscription (confirmed absent here, since $16 \ll 32$ available cores). This illustrates a practical limitation of simple two-parameter Amdahl's Law fits: they lump growing communication/synchronisation overhead together with true fixed serial work, which can make an otherwise near-perfectly-parallel algorithm look like it has a larger "serial fraction" than it actually does.

3. Interprocess Communication Time

3.1 Measured communication cost (leapfrog seed distribution)

numproc	messages sent (p-1)	avg comm_time (s)	est. time per message
2	1	6.96×10^{-4}	$\sim 7.0 \times 10^{-4}$ s
4	3	1.52×10^{-4}	$\sim 5.1 \times 10^{-5}$ s
8	7	1.95×10^{-4}	$\sim 2.8 \times 10^{-5}$ s
16	15	1.60×10^{-4}	$\sim 1.1 \times 10^{-5}$ s

Communication time is on the order of **tens to hundreds of microseconds in total**, and — critically — it does **not** scale with N. The leapfrog method sends only **p-1 messages total**, once at start-up, regardless of whether N is 10,000 or 10 billion. This is completely negligible next to compute times of hundreds of milliseconds to over a second.

3.2 Estimated time for a "master generates and sends every number" approach

If, instead of leapfrog, the master alone generated all $N = 100,000,000$ random numbers and sent each one individually to its corresponding slave, the number of messages required would jump from **O(p)** to **O(N)**.

Using the smallest (most steady-state) measured per-message cost from the table above, $\approx 1.1 \times 10^{-5}$ s/message (p=16 case, where fixed per-run overhead is amortised over the most messages):

```
Estimated time  $\approx N \times (\text{time per message})$   

 $\approx 100,000,000 \times 1.1 \times 10^{-5}$  s  

 $\approx 1,100$  seconds (~18 minutes)
```

Using the more conservative (worst-case, p=2) per-message cost of $\approx 7 \times 10^{-4}$ s instead:

Estimated time $\approx 100,000,000 \times 7 \times 10^{-4} \text{ s} \approx 70,000 \text{ seconds}$ (~19.4 hours)

Either bound — **from ~18 minutes to nearly 20 hours** — this estimate does **not** include:

- The master having to generate all N numbers **serially** first (LCG generation is inherently sequential, so this cannot be parallelised in a master-generates-all design).
- Slaves sitting idle waiting to receive one number at a time rather than computing continuously.
- Network contention from $N/(p-1)$ messages per slave arriving in a tight burst, which typically makes per-message overhead worse at scale, not better.

Comparison: the current leapfrog-based program completes the entire computation (setup + compute + reduce) in under 1.5 seconds even at numproc=1, and under 0.2 seconds at numproc=16. A master-sends-every-number design would require, conservatively, **thousands of times longer** purely for message passing overhead. This makes concrete the qualitative argument from the lecture notes: distributing only the initial seeds ($O(p)$ messages) and letting each process independently "leapfrog" through its own subsequence is dramatically more communication-efficient than distributing every individual random number ($O(N)$ messages).

4. Summary

- **Accuracy:** The Monte Carlo estimate of $\Gamma(5) = 24.0026$ vs. the true value 24 (0.01% relative error) confirms the leapfrog-based random number generation is statistically correct across all tested process counts.
- **Speed-up:** Close to ideal for $p \leq 4$ (efficiency $\geq 94\%$), consistent with the embarrassingly parallel nature of Monte Carlo integration. Efficiency declines progressively to 81.3% at $p=8$ and 49.2% at $p=16$, attributable to `MPI_Reduce` overhead growing with process count (from 0.002% of runtime at $p=1$ to 21.3% at $p=16$) rather than to a true fixed serial fraction or hardware oversubscription (32 cores were available; only 16 were used).

- **Communication:** The leapfrog method's $O(p)$ message count keeps communication overhead in the microsecond-to-sub-millisecond range and independent of N , versus an estimated 18 minutes to ~20 hours for a naive master-sends-every-number design with $O(N)$ messages — a difference of roughly three to four orders of magnitude.